TurboApply
The Architectural Evolution and Practical Implementation of a Job Recommendation Tool

Keywords: Job Search Efficiency · Rule-Based Recommendation · Anti-Scraping Strategies · Cross-Platform Integration · Workflow Automation

1. Background and Pain Points

For heavy users of online job platforms, the biggest challenge is rarely **a lack of jobs to apply for**. The real challenge is navigating an overwhelming volume of listings without losing disproportionate amounts of time and energy.

Interestingly, this problem does not necessarily exist at the beginning. In fact, it tends to **emerge and intensify as usage becomes more frequent and sophisticated**.

1.1 The Initial State: The Golden Period

When you first start using LinkedIn, the experience feels almost effortless. Simply enter **job title + choose Ireland + choose posted in the past 24 hours**, and nearly every listing on every page looks like a worthwhile opportunity. That sense of **opening a search and finding that almost every result is relevant** is the ideal state every job seeker hopes to achieve.

1.2 The Three Challenges That Emerge with Deeper Usage

However, as your use of the platform becomes more intensive over time, **three fundamental challenges inevitably begin to emerge:**

(1) Challenge #1: Persistent Sponsored Listings Can Dilute the Signal

After scrolling through several pages of results, candidates may find that a significant proportion of listings are long-running or sponsored positions from major companies. To be clear, this is not inherently problematic — it is a reasonable and mutually beneficial model for both employers and platforms.

From the candidate's perspective, however, there is an opportunity to make the discovery experience even more efficient. More intelligent, candidate-oriented filtering could help surface the most relevant opportunities while preserving the value that sponsored listings provide to employers and platforms. Ultimately, the ideal outcome is a win-win-win for candidates, employers, and platforms.

These positions are not necessarily fake or irrelevant. The challenge is more about timing and relevance: at a particular point in time, only one or two of these listings may represent genuinely strong opportunities for a given candidate.

As a result, the application process can become less efficient. A candidate may browse through five or six pages of listings, yet ultimately identify only a handful of opportunities that are highly relevant and worth applying to.

(2) Challenge #2: Duplicate Applications Consume More Than Just Time

An even more subtle challenge is duplicate applications. You may have applied for a position just a few days ago, only to see the same role — or a highly similar one — reappear in your search results with a “Posted in the past 24 hours” label.

It appears to be a fresh opportunity, and because the role genuinely looks like a strong fit, you may apply

again — only to realize later that you have submitted multiple applications to the same company for essentially the same position.
The challenge here is not simply the additional time spent applying. It is also the opportunity cost: time spent reviewing and re-evaluating a position you have already considered could instead have been used to discover genuinely new opportunities.
On the other hand, if you decide not to apply, you still need to keep scrolling and searching to determine what else is available.
Over time, this creates a familiar pattern: a significant amount of effort goes into searching and screening, while the number of genuinely meaningful applications remains relatively small.

(3) Challenge #3: Cross-Platform Expansion Amplifies the Problem Rather Than Solving It

Once you recognize the limitations of relying on a single platform, the natural response is to expand your search to additional platforms such as Indeed and IrishJobs in an attempt to broaden your coverage.
But instead of discovering a "new continent," you may encounter many of the same challenges across different platforms — duplicated listings, persistent postings, information overload, and the same time-consuming manual screening process.
In other words, adding more platforms can certainly increase the volume of opportunities you can access, but it does not necessarily increase the number of genuinely new and relevant opportunities you can act on.
This creates an interesting gap between search coverage and actionable opportunity coverage: more data does not automatically translate into more meaningful opportunities.

1.3 The Starting Point of the Tool

This tool was not built to help you find more jobs. It was built to help you decide faster which jobs are actually worth applying for — for you, at this particular point in time.
Its core objective is to restore the "initial-state" experience for serious, high-intensity job seekers:
Open the results table, and immediately see the jobs you should be applying for today.
Click the URL. Review the recommendations. Apply. Done.
No endless scrolling. No second-guessing. No wondering whether you've already applied.
Just the same efficient, focused, and frictionless experience you had when you first started using the platform — while continuously adapting to the reality of a much larger and more dynamic job market.

2. Core Philosophy: Rule-Based Recommendation Can Still Be High-Value

In the field of recommendation systems, there is a long-standing misconception that a recommendation system only becomes a "real" recommendation system once it incorporates ML or deep learning.
My practical experience has led me to a more pragmatic conclusion: in many recommendation scenarios, well-designed statistical rules grounded in an understanding of user behavior and decision-making can capture a large majority of the high-value recommendations. ML/DL models can then be introduced where they have the potential to deliver meaningful incremental improvements.
The key question, therefore, is not simply "Can we use ML?", but rather "Is the additional complexity justified by the value it delivers?"
Whether that marginal gain is worth pursuing ultimately comes down to a cost-benefit trade-off between engineering effort, system complexity, maintainability, and incremental recommendation quality.
The defining characteristic of this tool is therefore not simply that it is a "recommendation tool." Its recommendation logic is fundamentally rule-based, lightweight, and tailored to individual needs.
Its real value lies elsewhere: designing a lightweight and scalable solution that can be layered on top of existing job platforms, tools, and resources to significantly improve the efficiency of the job-search process.

This also reflects a broader engineering principle behind the project:
Use the simplest approach that can reliably solve the problem, and introduce additional complexity only when the incremental value justifies it.
With this principle in mind, I will next walk through the tool's overall architecture, the evolution of its design, and how it is used in practice.

3. Overall Architecture

The core data flow of the tool can be divided into five stages:

(1) Data Acquisition

Collect incremental data from job platforms to retrieve the latest job postings while minimizing unnecessary reprocessing of previously collected data.

(2) Historical Data Loading

Load the user's historical application records and reconcile them with the corresponding platform data.
At this stage, Jaro-Winkler similarity (jellyfish.jaro_winkler_similarity) is used for company-name matching. This helps bridge minor naming differences between personal records and platform data. For example, a company may be recorded manually as “ABC Company” in personal application records while appearing on the platform as “ABC Company Ireland.”

(3) Data Preprocessing & Feature Extraction

Clean, parse, normalize, and structure both platform data and historical application records to produce the features required for downstream filtering and recommendation.
This stage effectively transforms heterogeneous raw job data into a consistent, structured representation that the rule engine can reason over.

(4) Rule-Based Filtering & Recommendation Generation

Apply customizable, user-specific rules based on historical application behavior to filter opportunities and generate personalized recommendations.
The rule engine is designed to be extensible and configurable, allowing recommendation logic to evolve as the user's preferences, application strategy, and job-search context change.

(5) Application Tracking & Master Dataset Maintenance

Track application outcomes and append newly processed applications to the historical master dataset, ensuring that the system maintains a continuously updated record of application history.
This creates a closed feedback loop: historical application data influences recommendations, new applications generate additional historical data, and that updated history informs future recommendation decisions.
The overarching design goal is simple:
Based on my personal experience, turn a process where only 1–2 applications can be meaningfully completed per hour into one where 5–7 highly targeted applications can be submitted within the same hour.
This is not about maximizing the number of applications. It is about maximizing the number of relevant applications per unit of time.
In other words, the objective is not higher application volume for its own sake, but higher-quality decision-making and greater application efficiency.

4. Evolution of the Solution: From Full Automation to a Pragmatic Semi-Automated Approach

4.1 Initial Design: Full Automation (V1)

Our initial goal was to build a **fully automated, zero-touch workflow**, covering the entire process from data acquisition to job recommendation.
The workflow consisted of the following steps:

(1) Automatically scrape incremental job data

Automatically scrape incremental job data from LinkedIn and IrishJobs, retrieving relevant listings sorted by posting time in descending order so that the most recently published positions are processed first.

(2) Load historical application data

Load the user's historical application records.

During the initial setup, the historical dataset requires one-time preprocessing. Once the processed dataset has been saved, subsequent runs can directly load the existing results without repeating the preprocessing step.

(3) Append the previous day's actual applications to the master dataset

Append newly submitted applications from the previous day to the master dataset.

This step is necessary to update the key feature **last_apply_date** for each company, which is subsequently used by the recommendation logic.

(4) Preprocess newly scraped data and extract key features

Clean and transform the newly scraped data, then extract the features required by the downstream recommendation logic.

(5) Apply recommendation rules

Apply the recommendation rules to filter the candidate pool and generate the final recommendation dataset for the day.

(6) Export today_recommendation.csv

Export the daily recommendations to today_recommendation.csv, allowing the user to open the file and navigate directly to the corresponding application pages with a single click.

Under ideal conditions, this workflow was smooth and highly efficient.

But real-world operation quickly exposed an important constraint.

4.2 Real-World Bottleneck: Platform Access Constraints

In practice, the fully automated scraping approach encountered significant access limitations on the two major platforms:

Platform	Observed Access Constraints
LinkedIn	Maximum of 10 job postings per page, with approximately 4 pages accessible through the crawler
IrishJobs	Maximum of 4 pages could be scraped; attempting to access a 5th page resulted in access denial, while repeated attempts could trigger IP-level restrictions

When the number of newly posted jobs on a given day is relatively small, these constraints are manageable. However, when the volume of new postings increases, the crawler may fail to capture the complete set of relevant listings.

This creates a fundamental trade-off:

The more we rely on automated crawling, the more the system's coverage becomes dependent on platform-specific access constraints.

For a recommendation system whose primary value depends on **comprehensive opportunity coverage**, missing a meaningful portion of the available data can significantly reduce the quality and versatility of the final recommendations.

4.3 Solution Adjustment: Manual HTML + Automated Parsing (V2)

Rather than continuously increasing the complexity of the crawler to work around platform-specific restrictions, we made a pragmatic engineering decision:

Replace fully automated crawling with a semi-automated workflow: manual HTML download + automated parsing and recommendation.

The revised workflow consists of three main steps:

(1) Manual HTML download

After filtering relevant jobs by position and location and sorting by posting date, save the result pages directly as HTML files using a consistent naming convention such as page_1.html, page_2.html, and so forth.

The files are stored in the corresponding xxx_raw_data directory. For example, LinkedIn pages are stored under linkedin_raw_data.

(2) Automated parsing

Invoke the platform-specific parsing script to process the downloaded HTML files and extract the required job information.

The downstream preprocessing, feature extraction, filtering, and recommendation logic remain fully automated.

(3) Cross-platform merging

Merge incremental datasets from different platforms into a unified dataset, with built-in deduplication logic to identify identical company–job postings appearing across multiple platforms.

4.4 Why V2 Is Actually a Better Engineering Trade-Off

At first glance, this adjustment may appear to be a **"step backward"** because one manual operation has been reintroduced.

In practice, however, it significantly improves the system's **versatility, stability, portability, and scalability**.

The key advantages are:

- **Less dependent on platform-specific access policies.** The recommendation pipeline is no longer tightly coupled to the crawling limitations of a particular platform.
- **Platform-agnostic architecture.** Any job platform that allows the user to save search results as HTML can potentially be integrated into the same downstream pipeline.
- **Greater search flexibility.** The user is no longer constrained by a predefined crawler strategy. Different searches can be performed sequentially — for example, "Agentic AI", "Machine Learning", "Data Scientist", "Data Engineering", and so on — and the resulting pages can all be processed by the same system.
- **Lower maintenance overhead.** There is no need to continuously maintain increasingly complex crawler logic for each platform or respond to every change in platform-side access behavior.
- **Minimal manual overhead.** Each run requires only a few additional minutes to download the relevant HTML pages, while the downstream parsing, deduplication, filtering, and recommendation remain automated.
- **Greater operational convenience.** The tool can be used whenever and wherever the user wants to conduct a focused job-search session, without requiring the crawler itself to remain continuously operational.

Most importantly, this adjustment separates **data acquisition** from **data processing and recommendation**. The system no longer depends on a tightly coupled crawler for every platform. Instead, any platform can become a data source as long as its search results can be captured as HTML.

With **company + position** serving as the core identity for deduplication, listings from different platforms can be consolidated into a unified opportunity pool while avoiding duplicate recommendations.

This ultimately transforms the tool from a **platform-specific crawler** into a more **portable, extensible job-search intelligence layer** that can sit on top of multiple existing job platforms.

5. Recommendation Rules in Detail

The system currently incorporates three core recommendation rules, all built around two key dimensions: **time and historical application behavior**.

These rules are intentionally **simple, transparent, and configurable**. They can be modified or extended according to the user's application strategy and changing preferences.

Rule	Condition	Recommendation Level	Design Logic
1	Posted within the last 7 days + Never applied to this company before	Must-apply	Recently posted opportunities from companies with no previous application history represent genuinely new opportunities and therefore receive the highest recommendation priority.
2	Posted within the last 3 days + More than 30 days since the last application to this company	Must-apply	A sufficiently long interval reduces the likelihood of repeatedly targeting the same company within a short period while allowing previously targeted companies to become relevant again when new opportunities emerge.
3	All recommended jobs sorted by posting time in descending order	Priority	More recently posted positions are generally prioritized because they tend to have a shorter time-to-application window and may offer a better opportunity to engage while the position is still actively being reviewed.

Key Design Details

There are a few important implementation details behind these rules:

- Today's recommendation table uses today's date as the provisional application date for all recommended jobs. This represents the assumption that the user intends to apply to the recommended positions today.
- The provisional date is not treated as a confirmed application. If the user decides not to apply to a particular position, the preset date can simply be removed. When the next update is appended to the master dataset, only positions that were actually submitted are recorded as completed applications.

This distinction is important because it allows jobs that were reviewed but not submitted to re-enter the recommendation pool in the future if they continue to satisfy the recommendation criteria.

- Applications submitted through other channels can also be incorporated manually. For example, if the user applies to a position through a job alert, referral, or another platform, the corresponding record can be manually added to the results table so that the master application history remains consistent.

The overall design principle is **deliberately straightforward**: Use recentness to identify opportunities, historical application behavior to control repetition, and posting time to prioritize the final recommendation list.

This keeps the recommendation logic transparent, explainable, and easy to customize, while avoiding unnecessary model complexity.

6. Observed Efficiency Improvement

The primary objective of the tool is not to maximize application volume, but to reduce the time spent on repetitive search, screening, and duplicate checking so that more time can be devoted to meaningful applications.

Based on my personal experience, the difference can be summarized as follows:

Metric	Using Job Platforms Directly	Using This Tool
Applications completed per hour	Approximately 1–2	Approximately 5–6
Cross-platform coverage	Primarily platform-by-platform, with potential duplication across platforms	Multi-platform integration with cross-platform deduplication
Recommendation process	Primarily dependent on manual screening and judgment	Consistent, rule-based filtering and prioritization
Application history management	Often requires separate manual tracking	Centralized historical application dataset

In my previous workflow, completing a single meaningful application could take approximately one hour, including searching, screening, checking previous applications, and navigating to the relevant application page.

After introducing this tool, I can typically complete 5–6 targeted applications within the same hour.

This represents a substantial improvement in application efficiency.

More importantly, the time saved does not need to be spent submitting even more applications. It can instead be redirected toward job research, interview preparation, personal projects, professional development, or simply rest and recovery.

The goal is therefore not: Apply to more jobs.

It is: Spend less time searching, and more time acting on the opportunities that actually matter.

7. User Guide

7.1 Target Audience

This tool is best suited for users who:

- Search for jobs on LinkedIn, IrishJobs, or other platforms but typically complete only 1–2 meaningful applications per hour;
- Want to systematically maintain their application history and reduce the risk of duplicate submissions;
- Use multiple job platforms simultaneously and want to consolidate opportunities and recommendations in one place.

⚠️ Who may benefit less?

If you are already able to consistently identify and submit 5–7 highly relevant applications per hour, your existing workflow is already highly efficient, and the marginal benefit of introducing this tool may be limited.

7.2 Environment Requirements

Dependency	Version
Python	3.12.7
Poetry	2.4.1

7.3 Installation and Execution

(1) Download the project

Download and extract the source code. Users without Git can download the project as a ZIP archive.

(2) Navigate to the project directory

cd your-project-directory

(3) Install dependencies

poetry install --no-root

(4) Prepare the raw HTML data

Search for relevant jobs on LinkedIn, IrishJobs, or other supported platforms as needed.

After applying your desired filters — such as position, location, and posting date — save the result pages as HTML files using the naming convention:

page_1.html

page_2.html

page_3.html

...

Place the files in the corresponding xxx_raw_data directories. For example:

linkedin_raw_data/

├──── page_1.html

├──── page_2.html

└──── ...

(5) Configure the execution parameters

Open one_click_run.sh in the project root directory and configure the following parameters according to the HTML files downloaded for the current run:

linkedin_start_page

linkedin_end_page

irishjobs_start_page

irishjobs_end_page

For example, if two LinkedIn pages were downloaded today:

linkedin_start_page=1

linkedin_end_page=2

(6) Run the pipeline

Execute: poetry run bash one_click_run.sh

The pipeline will then perform the downstream processing automatically, including parsing, preprocessing, cross-platform merging, deduplication, historical-data integration, filtering, and recommendation generation.

7.4 Accessing Recommendation Results

After execution completes, the following file will be generated in the project root directory:

today_recommendation.csv

Open the file and navigate to the url column.

Each URL provides direct access to the corresponding job posting or application page, allowing the user to move from recommendation → application with a single click.

8. Extensibility and Future Plans

8.1 Currently Implemented Extensibility

The current architecture is intentionally designed to remain lightweight while supporting future expansion:

- **Multi-platform integration:** New job platforms can be integrated by adding a platform-specific parser.py implementation, without fundamentally changing the downstream recommendation pipeline.

- **Cross-platform deduplication:** Built-in deduplication logic helps identify the same company–job opportunity across different platforms and prevents duplicate recommendations.
- **Persistent application history:** Historical application data is stored persistently and can be incrementally updated as new applications are submitted.

Together, these capabilities separate **platform-specific data acquisition** from the **platform-independent recommendation logic**, making the system relatively easy to extend.

8.2 Possible Future Improvements

One longer-term possibility is that some of these capabilities could eventually be incorporated directly into job platforms as an **optional, candidate-side value-added service**.

Rather than requiring candidates to build an external tool to manage search results, application history, duplication, and personalized filtering, platforms themselves could provide an intelligent layer that helps candidates identify the opportunities most relevant to them at a given point in time.

In that sense, the goal is not necessarily to replace existing job platforms, but to explore **how the candidate experience could become significantly more efficient on top of the infrastructure that already exists**.

9. Final Thoughts

Throughout the process of building this tool, one of my deepest realizations has been this:

The essence of efficiency is not doing more things. It is making fewer unnecessary decisions.

When the machine can take over the repetitive judgment of **"Is this job worth applying for?"**, the candidate can redirect that cognitive energy toward a much more valuable question:

"How can I make this application as strong as possible?"

That is ultimately what TurboApply is trying to achieve — not simply helping candidates **apply faster**, but helping them spend more of their limited time on the parts of the job-search process where human judgment matters most.

If you're also finding the job-search process increasingly time-consuming, perhaps this approach can offer another way to think about it.

10. Declarations

10.1 Architecture and GenAI Assistance

The overall architecture, recommendation strategy, and design evolution of this tool are based on my own hands-on experience and experimentation during an intensive job-search process.

At the implementation level, however, I also made extensive use of GenAI tools as development assistants:

- The platform-specific xxx_parser.py implementations were developed with assistance from **Claude Code**, which I found particularly effective for code generation, debugging, and iterative software development.
- The one_click_run.sh execution script was developed with assistance from **DeepSeek**, which was well suited to this relatively straightforward scripting and automation task.
- Chinese-language optimization was supported by **DeepSeek**, particularly because I found it effective at preserving context and nuance in Chinese.
- The initial English-language version was optimized with assistance from **DeepSeek**. While useful for translating and structuring the content, some expressions retained a more Chinese-influenced English style.
- The final English-language version was refined with assistance from **ChatGPT**, which I found particularly effective for producing more natural, globally oriented English and improving the overall consistency, tone, and readability of the document.

This experience also reinforced another observation: **different GenAI products have different strengths**. Rather than treating these tools as interchangeable, users can benefit from combining their respective strengths across different stages of a project. From a product perspective, these differentiated capabilities may also represent an important source of competitive advantage for GenAI companies — and a foundation from which they can continue to expand their capabilities.

10.2 Platform Compatibility

The current platform-specific parsers are fully functional with the versions of the target platforms used during development.

However, web platforms can change their page structures, access policies, or other implementation details over time. Future compatibility issues therefore cannot be ruled out.

When such changes occur, the parsing and integration logic can be updated with the assistance of modern GenAI development tools.

10.3 A Note to Fellow Job Seekers

Finally, I hope that everyone on the job-seeking journey can find a position they genuinely enjoy and feel proud of.

Stay strong, keep moving forward, and let's encourage one another along the way.

GitHub Repository

The project is now **open source**.

You are welcome to explore the code, use the tool, submit issues, or contribute improvements:

TurboApply_Lightweight_Tool on GitHub